도구 시스템과 코드를 바꾸는 메커니즘

모델이 제안한 구조화 호출이 어떻게 검증·권한·실행·관찰을 거쳐 실제 파일과 프로세스의 변화가 되는지 읽는다

이 권을 마치면 Tool을 "함수 호출"이 아니라 schema, 정책, 효과, 진행 사건, 결과 표현을 결합하는 실행 계약으로 설명할 수 있다.

STUDY SNAPSHOT
Claude Code v2.1.88
2ca5ddabfed5

제3자 복원 소스 기반
한국어 · Guided Edition

오늘 우리는 이 코드의 어느 부분을 함께 보는가

Claude Code의 결정적 차이는 답변을 생성하는 데서 끝나지 않고 저장소를 실제로 바꾼다는 데 있다. Tool.ts가 표현 언어를 정의하고, toolExecution이 공통 실행 절차를 보장하며, 개별 도구가 파일·shell·검색의 구체적 의미를 구현한다. 이 권은 그 결합 지점과 분리 가능한 지점을 모두 본다.

203	59,678	323	881
분류된 TS/TSX 파일	물리 코드 줄	async 함수	tool use 표기

CHAPTER 01

Tool 계약은 한 기능의 전체 생애를 담는다

입력 schema부터 화면 렌더링까지 한 객체에 모인 이유와 그 비용을 이해해야 새 도구를 안전하게 추가할 수 있다.

CHAPTER 02

공통 실행 파이프라인이 도구의 의미를 지킨다

도구마다 call 구현은 달라도 unknown 처리, 취소, hook, 권한, 결과 pairing은 같은 순서와 관찰 규칙을 가져야 한다.

CHAPTER 03

파일 도구는 관찰한 상태에 근거해 쓴다

Read와 Edit는 별도 편의 기능이 아니라 하나의 낙관적 동시성 계약을 이룬다.

CHAPTER 04

Shell과 검색 도구는 운영체제를 해석한다

BashTool은 문자열을 process에 넘기는 래퍼가 아니라 명령 의미, 경로, 권한, sandbox, streaming output을 중재하는 하위 시스템이다.

분포 수치는 중요도 점수가 아니다. 이 권이 직접 펼쳐보는 코드 표면의 규모와 언어 메커니즘 밀도를 가늠하기 위한 탐색 표지다.

01

Tool 계약은 한 기능의 전체 생애를 담는다

입력 schema부터 화면 렌더링까지 한 객체에 모인 이유와 그 비용을 이해해야 새 도구를 안전하게 추가할 수 있다.

이 장에서 실제로 펼쳐볼 경로

src/Tool.ts · src/tools.ts · src/types/permissions.ts

Tool 타입은 실행 가능한 capability descriptor다

name과 call만으로는 모델·정책·UI가 같은 도구를 이해할 수 없다.

`Tool` 타입에는 input/output schema, prompt, validation, permission matching, concurrency safety, call, render 함수가 모여 있다. 하나의 정의가 모델에게 보이는 계약과 runtime이 실행하는 계약을 연결한다.

이 집중은 registry drift를 줄인다. 별도 파일의 JSON schema와 별도 switch의 executor가 어긋나는 일을 막는다. 동시에 UI와 telemetry까지 끌어안으면서 interface가 넓어지고, 작은 도구도 많은 관심사를 알아야 하는 비용이 생긴다.

새 필드를 추가할 때 모든 tool이 정말 소유해야 하는지 묻자. 공통 lifecycle hook인지, 특정 adapter의 projection인지 구분하지 않으면 Tool 타입은 점점 application container가 된다.

Tool contract를 구성하는 네 층

분석용 TypeScript — 실제 구현의 제어 구조를 축약·재작성

```
type Tool<I, O> = {
  inputSchema: Schema<I>
  validateInput(input: I): Validation
  checkPermissions(input: I): Decision
  call(input: I, ctx: Context): Promise<O>
  renderResult(output: O): View
}
```

1. **Schema**
   unknown JSON을 I로 좁히는 runtime boundary다.
2. **Policy**
   형식이 맞는 입력이 현재 허가되었는지 별도 판단한다.
3. **Projection**
   model result와 human UI가 같은 O에서 다른 표현을 만든다.

언어·런타임 안쪽에서는 generic은 compile-time 관계를 보존하지만 JSON 입력을 검증하지 않는다. schema와 generic이 서로 다른 층을 담당한다.

스터디에서 확인할 질문

도구 정의에서 반드시 같은 owner가 가져야 할 필드와 별도 adapter로 분리할 필드를 나눠 보라.

buildTool은 런타임 기본값과 정적 추론을 합친다

작은 factory가 60개 이상의 구현에 공통 행동을 주면서 각 도구의 구체 타입을 보존한다.

buildTool은 defaults를 펼치고 정의를 덮어쓴다. generic BuiltTool<D>는 input/output schema에서 구체 타입을 추론해 call과 renderer 사이를 잇는다.

이 패턴은 class 상속보다 얕고 조합하기 쉽다. 그러나 마지막 type assertion은 runtime 검증이 아니라 컴파일러와의 계약이다. 복원 소스의 누락 때문에 전체 typecheck가 불완전하다면 assertion의 안전성을 독립적으로 확인해야 한다.

factory를 바꿀 때는 runtime default와 type-level default를 함께 시험하자. 기본 checkPermissions가 바뀌면 모든 도구의 보안 의미가 달라질 수 있다.

Generic factory가 정의 D의 구체 타입을 되돌려 주는 이유

분석용 TypeScript — 실제 구현의 제어 구조를 축약·재작성

```
01  function buildTool<D extends ToolDef>(def: D): BuiltTool<D> {
02    return {
03      ...defaults,
04      userFacingName: () => def.name,
05      ...def
06    } as BuiltTool<D>
07  }
```

1. **D extends**
   최소 shape를 요구하면서 literal name과 schema type을 잃지 않는다.
2. **spread order**
   def가 default를 덮어쓰므로 runtime 의미가 순서에 달려 있다.
3. **as BuiltTool<D>**
   컴파일러에게 주는 주장이지 실행 검사가 아니다.

언어·런타임 안쪽에서는 TypeScript structural typing과 conditional/inferred type은 factory 호출부의 구체 타입을 보존한다. 마지막 assertion의 타당성은 contract test가 보완해야 한다.

스터디에서 확인할 질문

default가 새 필드를 자동 제공하는 편의와 도구가 명시적으로 선택하게 하는 안전성 중 어느 쪽이 적절한가?

schema validation과 semantic validation은 다르다

문자열 필드가 존재한다는 사실과 그 파일을 지금 편집해도 된다는 사실은 별개다.

input schema는 형태와 기본 타입을 확인한다. `validateInput`은 old/new string 동일 여부, 파일 존재, read-before-write 상태, 특수 경로처럼 도구 의미를 검사한다. permission check는 다시 사용자가 그 효과를 허용했는지를 판단한다.

세 검사를 하나로 합치면 오류 메시지는 간단해 보여도 authority가 흐려진다. 형식 오류를 사용자에게 허가받을 수는 없고, schema가 맞는다고 안전한 것도 아니다.

개별 도구를 리뷰할 때 검사를 표로 나누자: representable, meaningful-now, authorized. 각 검사가 어느 상태를 읽으며 실패 시 어떤 result를 만드는지 보면 누락과 중복이 드러난다.

스터디에서 확인할 질문

존재하지 않는 파일에 대한 Write와 Edit은 같은 schema를 가져도 semantic validation이 왜 달라야 하는가?

registry는 모델이 사용할 수 있는 언어의 어휘다

도구 파일이 존재해도 현재 permission context와 mode에서 pool에 들어가지 않으면 모델에게는 없는 기능이다.

`getTools`와 pool 조립 코드는 preset, simple mode, deny rule, MCP 도구를 결합한다. 이것은 import 목록이 아니라 현재 세션이 사용할 수 있는 capability set을 계산하는 과정이다.

deny된 도구를 모델에게 보여 준 뒤 매번 거부하는 것과 처음부터 목록에서 빼는 것은 모델 행동과 token 사용이 다르다. 현재 코드는 blanket deny를 조기에 필터링해 불가능한 행동을 제안할 확률을 낮춘다.

동적 registry를 개선할 때 discovery와 authority를 섞지 말자. MCP server가 도구를 광고했다는 사실은 그 도구의 실행 허가가 아니다.

스터디에서 확인할 질문

tool list가 turn 중 바뀌면 모델에게 이전 목록으로 생성된 호출을 어떻게 처리해야 하는가?

02

공통 실행 파이프라인이 도구의 의미를 지킨다

도구마다 call 구현은 달라도 unknown 처리, 취소, hook, 권한, 결과 pairing은 같은 순서와 관찰 규칙을 가져야 한다.

이 장에서 실제로 펼쳐볼 경로

src/services/tools/toolExecution.ts · src/services/tools/toolOrchestration.ts

unknown tool도 protocol-valid result가 되어야 한다

찾지 못했다는 runtime 오류를 다음 모델이 이해할 수 있는 관찰로 바꾼다.

`runToolUse`는 먼저 현재 pool에서 이름을 찾고, 오래된 transcript를 위한 alias fallback을 확인한다. 그래도 없으면 error tool_result를 같은 id로 만들어 yield한다.

여기에는 호환성과 protocol 보존이 함께 있다. alias는 과거 이름을 살리지만 모든 base tool로 무제한 fallback하지 않아 현재 deny를 우회하지 않게 범위를 제한한다.

도구 rename을 설계할 때 source alias, model prompt, persisted transcript, analytics name을 함께 봐야 한다. 문자열 하나만 바꾸면 재개 세션이 깨질 수 있다.

스터디에서 확인할 질문
deprecated alias를 언제 제거할 수 있으며 그 결정을 뒷받침할 session-version evidence는 무엇인가?

실행 전 단계의 순서는 보안 의미를 갖는다

backfill, hook, validation, permission이 input을 읽고 바꿀 수 있으므로 어느 버전을 다음 단계가 보는지가 중요하다.

toolExecution은 observable input을 보강하고, hooks가 updatedInput을 낼 수 있으며, permission decision도 input을 수정할 수 있다. 마지막 call은 원본이 아니라 승인된 processedInput을 받아야 한다.

이 pipeline에서 한 단계가 fresh object를 반환하고 다른 단계는 in-place mutation을 기대하면 audit와 transcript hash가 엇갈린다. 코멘트가 clone과 mutation ownership을 세밀하게 다루는 이유다.

리팩터링 시 "validation을 앞당기자" 같은 변경은 성능 조정이 아니라 의미 변경이다. 어떤 변환 전후의 input을 검사하는지와 사용자가 무엇을 승인했는지 재검토해야 한다.

processedInput이 만들어지는 실제 pipeline

01	02	03	04	05
Parse	**Backfill/Hook**	**Validate**	**Permission**	**Call**
schema로 unknown을 typed input으로 좁힌다.	경로 보강과 허용된 input 수정	현재 상태에서 의미가 있는지 검사	승인된 최종 input을 결정	동일한 input으로 effect 실행

이 그림이 감추지 않는 것 각 단계가 새 object를 반환하는지 in-place로 바꾸는지가 중요하다. 사용자가 본 값과 call이 받은 값의 identity가 일치해야 한다.

스터디에서 확인할 질문

hook이 명령을 수정한 뒤 기존 permission allow rule을 그대로 적용해도 되는가?

함께 펼쳐볼 자료 src/services/tools/toolExecution.ts:735-940 · src/Tool.ts:460-520

진행 사건과 최종 결과는 다른 계약이다

오래 걸리는 shell·agent 작업은 완료 전에도 관찰 가능해야 하지만 중간 상태를 성공으로 오해하면 안 된다.

tool call은 progress callback과 generator update를 통해 중간 출력을 전달한다. 최종 `ToolResult`에는 model-facing content와 UI/telemetry에 필요한 structured data가 함께 들어갈 수 있다.

progress는 latency 경험을 개선하지만 durable truth는 아니다. process가 죽으면 마지막 progress 이후 실제 상태를 알 수 없고, SDK 소비자가 일부 event를 놓칠 수 있다. 완료 event와 settlement를 별도로 정의해야 한다.

새 진행 event를 추가할 때 빈도 제한, 순서, 취소 후 발행 여부를 정하자. 너무 세밀한 output은 terminal 렌더와 네트워크 소비자 모두에 backpressure를 만든다.

스터디에서 확인할 질문

같은 tool call에서 progress 순서와 final result 순서를 보장하려면 어떤 producer ownership이 필요한가?

동시성 안전성은 도구 이름이 아니라 구체 입력의 속성이다

Read라는 이름도 외부 상태를 건드릴 수 있고 Bash라는 이름도 순수 조회일 수 있다.

`partitionToolCalls`는 schema parse 후 각 도구의 `isConcurrencySafe(input)`를 호출하고, 연속된 안전 호출만 batch로 묶는다. 판정이 실패하면 보수적으로 직렬 실행한다.

이 설계는 read/write 도구의 고정 목록보다 정교하다. 같은 BashTool도 `git status`와 `rm`은 다르다. 그러나 판정 함수가 실제 effect와 불일치하면 race가 생기므로 correctness-critical policy다.

병렬 batch의 context modifier를 도구 완료 뒤 결정적 순서로 적용하는 것도 중요하다. 실행 완료 순서가 session state 순서를 임의로 바꾸지 않게 한다.

연속 tool call의 batch 분할



`safe + previous safe` 같은 read batch에 추가

`unsafe` 단독 serial batch 생성

`parse/analysis error` 보수적으로 unsafe

`batch done` modifier를 deterministic order로 적용

언어·런타임 안쪽에서는 Promise 병렬성보다 중요한 것은 shared state에 대한 commutativity다. read-only라는 이름이 아니라 구체 effect가 순서 독립적인지 판단한다.

스터디에서 확인할 질문

두 read-only tool이 같은 cache나 rate limit을 갱신한다면 여전히 concurrency-safe라고 할 수 있는가?

03

파일 도구는 관찰한 상태에 근거해 쓴다

Read와 Edit는 별도 편의 기능이 아니라 하나의 낙관적 동시성 계약을 이룬다.

이 장에서 실제로 펼쳐볼 경로

src/tools/FileReadTool/** · src/tools/FileEditTool/** · src/tools/FileWriteTool/**

Read는 출력과 함께 편집 전제를 만든다

파일 내용을 모델에 보여 주는 동시에 timestamp·범위·content를 readFileState에 남긴다.

FileReadTool은 text, image, notebook 등 형식을 구분하고 offset/limit 및 token 상한을 적용한다. 큰 파일을 무제한 prompt에 넣지 않으면서도 모델이 읽은 범위를 추적한다.

이 cache는 성능 캐시만이 아니다. Edit가 "마지막으로 읽은 뒤 바뀌지 않았는가"를 판단하는 증거다. 부분 read와 전체 read를 구분하는 이유도 stale 판정의 확실성이 다르기 때문이다.

Read 결과 형식을 바꿀 때 UI와 모델 가독성뿐 아니라 Edit가 의존하는 관찰 상태가 그대로 갱신되는지 확인해야 한다.

스터디에서 확인할 질문

부분 범위만 읽은 파일을 편집하도록 허용할 때 어떤 변경은 안전하고 어떤 변경은 추가 read가 필요한가?

Edit는 read-before-write를 낙관적 잠금으로 사용한다

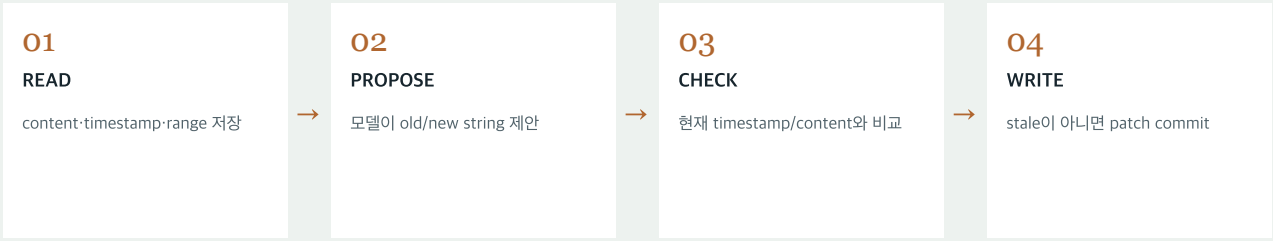
파일을 잠그지 않고도 사용자가 바꾼 내용을 덮어쓰지 않기 위해 마지막 관찰과 현재 상태를 비교한다.

call 단계는 현재 파일을 동기로 읽고 modification time을 readFileState와 비교한다. full read였다면 timestamp가 달라도 content가 같은 경우 false positive를 줄인다. 다르면 `FILE_UNEXPECTEDLY_MODIFIED_ERROR`로 중단한다.

이는 optimistic concurrency control과 닮았다. 오래 잠금을 잡지 않고 commit 직전에 version을 확인한다. 다만 filesystem timestamp와 process-local cache에 의존하므로 완전한 transaction이 아니다.

이 메커니즘을 제거하고 "항상 최신 파일에서 old_string 검색"만 하면 사용자의 동시 변경 일부를 우연히 보존할 수 있지만 모델이 읽지 않은 문맥에 쓰게 된다. 안전성의 기준은 patch 적용 가능성이 아니라 관찰 근거다.

FileReadState가 version token으로 쓰이는 방식



이 그림이 감추지 않는 것 lock을 오래 잡지 않고 commit 직전에 version을 비교하는 optimistic concurrency다. timestamp는 약한 token이라 full content fallback이 보완한다.

스터디에서 확인할 질문

read state의 version token을 content hash로 바꾸면 정확성·비용·대용량 파일에서 어떤 tradeoff가 생기는가?

critical window 안의 await 하나가 TOCTOU를 다시 연다

stale check와 write 사이에 scheduler가 개입하면 다른 편집이 끼어들 수 있다.

FileEditTool은 directory 생성과 history backup 같은 await를 미리 끝낸 뒤, 동기 read·검사·patch·write를 연속 수행한다. 주석이 이 구간에 async operation을 넣지 말라고 명시한다.

Node의 한 thread가 모든 race를 없애지는 않는다. await가 event loop에 제어를 돌려주면 다른 task나 외부 process가 파일을 바꿀 수 있다. await를 없애도 외부 process race는 남지만 내부 interleaving 창은 줄어든다.

리팩터링으로 편리한 async helper를 넣기 전에 이 구간의 시간적 불변조건을 테스트로 고정하자. 구조적으로는 versioned write나 atomic rename 같은 더 강한 primitive가 가능한지도 검토할 수 있다.

Event loop 관점의 Edit 임계 구간

분석용 TypeScript — 실제 구현의 제어 구조를 축약·재작성

```
01  await prepareDirectoryAndHistory()
02
03  const current = readSynchronously(path)
04  assertNotStale(current, lastRead)
05  const next = applyPatch(current, edit)
06  writeSynchronously(path, next)
07
08  notifyLspLater()
```

1. **await before**
   긴 준비 작업은 stale check 전에 끝내 내부 interleaving 창을 줄인다.
2. **sync window**
   같은 JS thread의 다른 task에 제어를 넘기지 않는다.
3. **external race**
   다른 process는 여전히 파일을 바꿀 수 있으므로 완전한 transaction은 아니다.

언어·런타임 안쪽에서는 await는 continuation을 microtask/event-loop에 예약한다. check와 use 사이 await를 넣으면 다른 task가 동일 state를 바꿀 기회가 생긴다.

스터디에서 확인할 질문

검사 직후 외부 process가 파일을 바꾸는 race까지 막으려면 OS 수준에서 어떤 원자적 조건부 write가 필요한가?

쓰기 뒤에는 다른 관찰계를 갱신해야 한다

disk write 성공만으로 LSP, file history, diagnostics, transcript가 새 상태를 아는 것은 아니다.

Edit는 파일을 쓴 뒤 LSP change/save 알림, diagnostics 정리, history update를 이어 수행한다. 일부는 fire-and-forget이므로 write의 성공과 후속 projection의 성공이 분리된다.

이 구조는 파일이 canonical effect이고 LSP는 파생 projection임을 보여 준다. 그렇다고 projection 실패가 무시 가능한 것은 아니다. 사용자에게 오래된 diagnostics를 보여 주면 다음 판단이 틀릴 수 있다.

후속 작업을 transaction처럼 모두 묶기보다 각 실패의 recovery를 정의해야 한다. LSP 알림은 재동기화할 수 있고, history backup은 write 전에 확보해야 하며, transcript는 실제 write 결과를 반영해야 한다.

스터디에서 확인할 질문

파일은 바뀌었지만 LSP 알림이 실패한 경우 tool result를 성공으로 표시하면서 어떤 warning을 남겨야 하는가?

04

Shell과 검색 도구는 운영체제를 해석한다

BashTool은 문자열을 process에 넘기는 래퍼가 아니라 명령 의미, 경로, 권한, sandbox, streaming output을 중재하는 하위 시스템이다.

이 장에서 실제로 펼쳐볼 경로

src/tools/BashTool/** · src/tools/GrepTool/** · src/tools/GlobTool/** · src/tools/LSTool/**

BashTool의 크기는 shell 문법의 외부성을 반영한다

하나의 command 문자열에는 파이프, redirect, substitution, environment, compound operator가 들어갈 수 있다.

BashTool은 input validation, permission rule 추출, sandbox 선택, process 실행, output streaming과 background 전환을 다룬다. call 한 줄로 보이는 효과 뒤에 parser와 policy가 필요한 이유다.

shell은 이미 하나의 프로그래밍 언어이므로 문자열 prefix 비교만으로 effect를 분류할 수 없다. `echo`도 redirect와 command substitution을 통해 write나 실행을 만들 수 있다.

도구를 단순화하려면 기능을 없애는 것이 아니라 shell이라는 넓은 language surface를 더 작은 capability로 나누는 선택도 있다. 예를 들어 git query, package test, generic shell은 서로 다른 policy를 가질 수 있다.

스터디에서 확인할 질문

사용자 편의와 분석 가능성 사이에서 generic BashTool을 유지할 최소 이유는 무엇인가?

권한 패턴은 command의 의미를 근사한다

완전한 shell 해석 없이도 위험한 부분을 보수적으로 찾아야 한다.

bashPermissions와 bashSecurity는 compound command를 나누고 prefix rule을 만들며 dangerous construct와 path를 검사한다. 분석 실패나 subcommand 수 초과를 안전하다고 간주하지 않고 보수적 경로로 보낸다.

이는 정적 분석의 soundness/precision tradeoff다. 너무 느슨하면 우회가 생기고, 너무 엄격하면 harmless command도 매번 사용자에게 묻는다. 편의 때문에 fallback을 allow로 두는 순간 보안 경계가 사라진다.

새 shell 문법 지원을 추가할 때 parser test뿐 아니라 permission rule canonicalization과 adversarial example을 함께 추가해야 한다.

스터디에서 확인할 질문

`git -C dir status`와 `cd dir && git status`를 같은 permission 의미로 정규화할 수 있는가?

검색 도구는 범용 shell을 좁은 read capability로 바꾼다

Glob과 Grep은 입력 공간과 결과 형식을 제한해 모델과 권한 시스템이 더 정확히 추론하게 한다.

전용 검색 도구는 pattern, path, limit를 schema로 받고 structured result를 반환한다. 같은 일을 shell의 `find | xargs grep`로 할 수 있어도 permission과 portability, output truncation은 훨씬 다르다.

좁은 도구가 많아지는 비용은 registry와 prompt의 크기다. 따라서 capability를 쪼갤 때는 보안·정확성·관찰성 이득이 model selection 복잡성을 이기는지 보아야 한다.

전용 도구의 구현은 내부에서 ripgrep 같은 외부 binary를 쓸 수 있지만, adapter가 exit code와 platform 차이를 내부 result로 정규화해야 한다.

스터디에서 확인할 질문
어떤 작업을 새 전용 도구로 승격할지 판단하는 rubric을 effect 범위, 빈도, portability로 만들어 보라.

도구 생태계의 구조 개선은 contract test에서 시작한다

개별 도구 테스트만으로 공통 pipeline과 registry의 조합 오류를 잡기 어렵다.

도구마다 schema-valid/invalid input, permission allow/ask/deny, abort, success, exception을 같은 contract suite로 돌릴 수 있다. 추가로 read-only 판정과 result pairing을 공통 invariant로 검사한다.

이런 테스트는 구현을 동일하게 만들지 않는다. 각 도구의 semantic validation과 effect는 다르지만, runtime이 요구하는 최소 프로토콜을 만족하는지 확인한다.

구조를 domain/tool adapter로 나누고 싶다면 먼저 contract suite를 만들어 이동 전후 trace를 비교하자. 그렇지 않으면 넓은 interface를 분해하는 동안 숨은 default가 사라진다.

스터디에서 확인할 질문

새 Tool implementation이 registry에 들어가기 전에 자동으로 검증할 수 있는 다섯 가지 공통 속성은 무엇인가?

코드 뒤에서 작동하는 언어·런타임 메커니즘

문법 이름을 외우기보다, 런타임이 실제로 무엇을 저장하고 언제 제어를 넘기는지와 이 프로젝트가 그것을 어떤 책임에 사용하는지를 함께 본다.

generic factory

언어가 제공하는 것
정의 값 D에서 schema와 method signature의 구체 타입을 추론한다.

이 저장소가 쓰는 방식
buildTool이 공통 default를 주면서 각 도구 input/output 타입을 보존한다.

놓치기 쉬운 함정
마지막 assertion은 runtime 증명이 아니며 default 변경은 전체 도구 의미를 바꾼다.

runtime schema

언어가 제공하는 것
unknown JSON을 실행 전에 성공/실패 result로 검증한다.

이 저장소가 쓰는 방식
모델이 낸 tool arguments를 semantic validation 전에 좁힌다.

놓치기 쉬운 함정
TypeScript 타입은 실행 중 사라지므로 schema 없이 외부 입력은 안전하지 않다.

async iterator composition

언어가 제공하는 것
하위 iterator의 사건을 상위 stream에 순서대로 전달한다.

이 저장소가 쓰는 방식
permission·call·progress 결과를 query까지 운반한다.

놓치기 쉬운 함정
중간 iterator가 throw/return할 때 final observation을 놓칠 수 있다.

input-dependent concurrency

언어가 제공하는 것
같은 함수라도 구체 인자에 따라 병렬 가능성을 다르게 판정한다.

이 저장소가 쓰는 방식
연속된 read-only tool call만 제한된 병렬 batch로 만든다.

놓치기 쉬운 함정
판정 실패를 안전으로 간주하면 shared state race가 생긴다.

optimistic concurrency

언어가 제공하는 것
읽은 version과 commit 직전 version을 비교해 stale write를 거부한다.

이 저장소가 쓰는 방식
Read cache와 Edit의 timestamp/content 검사가 사용자의 동시 변경을 보호한다.

놓치기 쉬운 함정
검사와 write 사이의 await 또는 외부 process race는 별도 문제다.

event-loop critical window

언어가 제공하는 것
동기 구간은 같은 thread의 다른 task에 제어를 넘기지 않지만 OS 외부 경쟁은 남는다.

이 저장소가 쓰는 방식
Edit가 stale check와 write 사이 await를 피한다.

놓치기 쉬운 함정
동기라는 이유만으로 filesystem transaction이라고 부를 수 없다.

스터디용 코드 탐색 인덱스

파일을 처음부터 끝까지 읽지 않는다. 아래 시작점에서 symbol을 잡고 정의·호출·상태·실패 경로를 왕복한다.

시작 경로	먼저 볼 표현	그 다음에 물을 질문
src/Tool.ts	Tool/ToolDef/BuiltTool	compile-time type와 runtime default가 어디서 만나는가
src/tools.ts	tool pool assembly	discovery와 authority가 분리되는가
src/services/tools/toolExecution.ts	processedInput pipeline	사용자가 승인한 정확한 값이 실행되는가
src/services/tools/toolOrchestration.ts	partitionToolCalls	병렬 판정과 context merge가 결정적인가
src/tools/FileReadTool/FileReadTool.ts	readFileState update	부분 read가 어떤 편집 전제를 남기는가
src/tools/FileEditTool/FileEditTool.ts	stale check/patch/write	임계 구간 안에 비동기 gap이 있는가
src/tools/BashTool/bashSecurity.ts	parse와 unknown 처리	실제 shell semantics와 어디까지 일치하는가
src/tools/GrepTool/GrepTool.ts	narrow schema/result	전용 도구가 generic shell보다 주는 이득은 무엇인가

코드 독해 규칙

타입 선언만 보고 runtime 검증을 가정하지 않는다. 호출 순서만 보고 authority를 가정하지 않는다. UI 표시만 보고 durable state를 가정하지 않는다. 주석만 보고 production 활성이나 upstream 의도를 확정하지 않는다.

Edit 한 번을 effect contract로 해부하기

모델의 JSON에서 disk write 이후 projection까지, 각 단계가 받는 input 버전과 실패 결과를 추적한다.

1. `FileEditTool`의 schema, semantic validation, permission, call을 서로 다른 색으로 표시한다.
2. `toolExecution`에서 hook과 permission이 input을 수정하는 지점을 찾고 최종 call input을 기록한다.
3. readFileState가 없는 경우, timestamp가 바뀐 경우, content만 같은 경우 세 반례를 비교한다.
4. write 성공 뒤 LSP·history·transcript 중 하나가 실패하는 상황을 가정해 settlement 표를 만든다.
5. 같은 분석을 BashTool 하나에 적용하고 파일 도구와 달라지는 authority·cancellation을 설명한다.

> **완료의 기준**
>
> Tool을 호출 가능한 함수가 아니라 representable → meaningful-now → authorized → effected → observed의 계약으로 설명할 수 있으면 된다.

다음 모임에 남길 기록

다음 모임에는 공통 Tool contract test 하나와 도구별 semantic test 하나를 의사코드로 준비한다.